

PANDUAN TEKNIS TUGAS MINGGU 2 — BACKEND RUANG DOSEN

Periode: 10 Mei — 16 Mei 2026

Tema: Security Implementation, Advanced Prisma Relations, & CRUD Optimization

🔗 Link Referensi Utama

- Repository GitHub: [Web-Ruang-Dosen \(https://github.com/BagasD-Tara/Web-Ruang-Dosen.git\)](https://github.com/BagasD-Tara/Web-Ruang-Dosen.git)
- Milestone Spreadsheet: [Google Sheets Milestone \(https://docs.google.com/spreadsheets/d/1Mcnbc4rKnNZQ69fxuaxOWTKug_8GJu3wJAUCtx6G0sY/edit?usp=sharing\)](https://docs.google.com/spreadsheets/d/1Mcnbc4rKnNZQ69fxuaxOWTKug_8GJu3wJAUCtx6G0sY/edit?usp=sharing)

⚠️ Protokol Wajib Sprint 2 (Harap Dibaca!)

1. Prosedur Git & Kolaborasi

- Update Repo:** Buka terminal, masuk ke folder project, pastikan berada di branch develop, lalu jalankan `git pull origin develop`. Lakukan ini setiap pagi sebelum mulai koding agar tidak tertinggal perubahan dari anggota tim lain.
- Format Branch:** Selalu buat branch baru untuk setiap tugas dengan format `fitur/backend-[nama]-[tugas]` (Contoh: `fitur/backend-sinta-course`).
- Pull Request:** Setelah selesai, push ke GitHub dan buat Pull Request. Target PR adalah branch develop. **WAJIB** Assign **Ariel (Leader)** sebagai Reviewer di GitHub.
- Merge:** Dilarang keras menekan tombol merge sendiri. Tunggu tim leader melakukan review kode Anda, memberikan approval, dan tim leader yang akan melakukan merge.

2. Aturan Prisma & Database

- Setiap kali Anda selesai melakukan edit pada file `schema.prisma`, Anda wajib menjalankan perintah `npx prisma generate` di terminal agar Prisma Client terupdate dengan skema terbaru.
- Jika perubahan skema Anda melibatkan penambahan tabel baru atau field baru, Anda wajib menjalankan `npx prisma migrate dev --name deskripsi_perubahan` agar database tereksekusi dan history migrasi tercatat.

3. Standar Clean Code

- Pastikan untuk memberikan penanganan error yang baik. Gunakan Exception bawaan NestJS seperti `NotFoundException` jika data tidak ditemukan, atau `ForbiddenException` jika user tidak memiliki akses.
- Sebelum melakukan commit, teliti kembali kode Anda. Hapus semua `console.log` yang digunakan untuk debugging dan hapus baris kode yang dikomentari atau tidak terpakai.
- Anda hanya diperbolehkan mengedit file di dalam folder modul Anda sendiri di bawah `apps/api/src/`. Jangan menyentuh kode milik modul anggota tim lain.

1. 👑 TUGAS ARIEL — Security Layer & Enrollment System

Branch: `fitur/backend-ariel-auth`

Deskripsi

Tugas ini berfokus pada pembangunan lapisan keamanan (Security Layer) untuk seluruh endpoint API kita. Anda harus mengimplementasikan Passport JWT sebagai "satpam" yang akan memverifikasi token setiap kali ada request masuk. Selain itu, Anda bertanggung jawab menyelesaikan logika pendaftaran (Enrollment), di mana mahasiswa dapat mendaftar masuk ke suatu mata kuliah.

Langkah-Langkah

A. Persiapan Awal

- Pindah ke branch utama (`git checkout develop`) lalu tarik update kode terbaru dari tim (`git pull origin develop`).
- Buat branch terpisah khusus untuk tugas Anda (`git checkout -b fitur/backend-ariel-auth`).
- Masuk ke folder backend (`cd apps/api`) lalu jalankan `npm install` untuk memastikan semua library/dependency terbaru terunduh dan sinkron.

B. Pembaruan Database (Prisma)

- Buka file `apps/api/prisma/schema.prisma`.
- Buat sebuah model baru bernama `Enrollment`. Model ini berfungsi sebagai tabel perantara (junction table) antara `User` dan `Course`.
- Di dalam model `Enrollment`, pastikan terdapat field ID (sebagai UUID), field untuk ID User, field untuk ID Course, relasi yang merujuk ke tabel User, relasi yang merujuk ke tabel Course, dan penanda waktu (`createdAt`).
- Pastikan Anda menambahkan aturan unik (unique constraint) kombinasi antara ID User dan ID Course di dalam model tersebut untuk mencegah satu mahasiswa mendaftar dua kali di kelas yang sama.
- Tambahkan relasi balikan (array of enrollments) di model `User` dan `Course`.
- Buka terminal, pastikan berada di `apps/api`, lalu jalankan `npx prisma migrate dev --name create_enrollment_table` untuk mengeksekusi tabel ini ke database sungguhan dan menyimpan histori migrasinya.

C. Mulai Coding (Security & Profile)

- JWT Strategy:** Buat file strategi JWT di dalam folder `src/auth/`. Di dalam file ini, buat class yang mewarisi strategi Passport. Konfigurasi strategi ini untuk membaca token dari header Authorization sebagai Bearer Token. Buat fungsi validasi untuk mengekstrak dan mengembalikan

`userId` dan `role` dari payload.

2. **Auth Guard:** Buat file guard kustom di folder yang sama, yang mewarisi guard JWT bawaan. Jika token tidak valid atau tidak ada, guard harus menolak dengan error 401 Unauthorized.
3. **Auth Module:** Buka file module auth dan pastikan kelas strategi JWT terdaftar di providers.
4. **Profile API (`auth.controller.ts`):**
 - Endpoint `GET /auth/profile`: Pasang guard JWT. Ekstrak `userId` dari request. Gunakan Prisma untuk mengembalikan data user. **ATURAN KETAT:** Pastikan response yang dikembalikan berisi tepat field ini: `nama`, `email`, `role`, jumlah XP, dan tanggal pendaftaran (`createdAt`).
 - Endpoint `PUT /auth/profile`: Pasang guard JWT. **ATURAN KETAT:** Field `email` dan `role` SAMA SEKALI TIDAK BOLEH diizinkan untuk diubah demi keamanan. Abaikan jika dikirim. Jika payload request mengandung `password` baru, pastikan Anda melakukan hashing menggunakan library `bcrypt` sebelum menyimpannya ke database.
5. **Enroll API (`course.controller.ts`):**
 - Buat endpoint `POST /courses/:id/enroll`.
 - **Validasi 1:** Gunakan Prisma untuk memastikan `courseId` yang dituju benar-benar ada. Jika tidak, return error 404 `NotFoundException`.
 - **Validasi 2:** Periksa apakah data pendaftaran mahasiswa tersebut di kelas ini sudah ada. Jika sudah, return error 400 `BadRequestException`.
 - Jika kedua validasi lolos, buat baris data baru di tabel `Enrollment`.

D. Uji Coba (Testing Postman)

1. Login dan salin token JWT.
2. Panggil `GET /auth/profile` dengan Bearer Token. Pastikan JSON menampilkan data nama, email, role, xp, dan tanggal daftar dengan akurat.
3. Coba panggil `PUT /auth/profile` dan paksa kirim perubahan email atau role. Pastikan sistem menolak perubahannya dan field tersebut tetap utuh.
4. Panggil `POST /courses/[ID_COURSE]/enroll`. Gunakan ID Course sembarang yang tidak ada di database, pastikan Anda mendapat error 404.
5. Gunakan ID Course valid, pastikan berhasil. Tembak sekali lagi, pastikan Anda mendapat error 400 karena duplikat.

E. Selesai? Push ke GitHub

1. Masukkan semua perubahan Anda: `git add .`
2. Simpan sejarah pekerjaan Anda: `git commit -m "feat: jwt guard, profile detail extraction, and robust enrollment validation"`
3. Unggah ke GitHub server: `git push origin fitur/backend-ariel-auth`
4. Buat Pull Request ke `develop` dan tugaskan Leader sebagai reviewer.

2. 👤 TUGAS SINTA— Course Management & Cascade Delete

Branch: `fitur/backend-sinta-course`

Deskripsi

Tugas Anda difokuskan pada pengelolaan entitas utama aplikasi, yaitu Mata Kuliah (Course). Anda harus menyempumakan fitur CRUD (Create, Read, Update, Delete) yang sudah ada dengan menambahkan pengambilan data relasional. Bagian terpenting dari tugas ini adalah memastikan integritas database terjaga melalui implementasi Cascade Delete, sehingga penghapusan suatu mata kuliah tidak menyisakan data yatim (orphaned data).

Langkah-Langkah

A. Persiapan Awal

1. Pindah ke branch utama (`git checkout develop`) lalu tarik update kode terbaru dari tim (`git pull origin develop`).
2. Buat branch terpisah khusus untuk tugas Anda (`git checkout -b fitur/backend-sinta-course`).
3. Masuk ke folder backend (`cd apps/api`) lalu jalankan `npm install` untuk memastikan semua library/dependency terbaru terunduh dan sinkron.

B. Pembaruan Database (Prisma)

1. Buka file definisi database di `apps/api/prisma/schema.prisma`.
2. Anda perlu memodifikasi model-model yang merupakan "anak" dari model `Course`: `Material`, `Quiz`, `Assignment`, `PracticalLab`, dan juga `Enrollment`.
3. Pada masing-masing model tersebut, temukan field relasi yang merujuk kembali ke tabel `Course`.
4. Tambahkan argumen penanda hapus kaskade (`onDelete: Cascade`) ke dalam definisi relasi tersebut. Ini menjamin jika `Course` dihapus, seluruh materi, kuis, tugas, praktikum, dan data mahasiswa yang terdaftar akan terhapus otomatis secara bersih.
5. Jalankan `npx prisma generate` di terminal untuk memperbarui Prisma Client sehingga kode Anda bisa membaca aturan relasi Cascade yang baru.

C. Mulai Coding (Logic Lanjutan)

1. Buka file service untuk course (`course.service.ts`).
2. **Fungsi `findAll`:** Modifikasi query agar melakukan penarikan data (`include`) nama dosen pengampu dari tabel `User`. Selain itu, wajib gunakan fitur agregasi Prisma (`count`) untuk menghitung total jumlah relasi pendaftaran (`Enrollment`) yang terhubung ke setiap course.
3. **Fungsi `findOne`:** Modifikasi query pengambilan detail course. Gunakan `include` untuk menarik daftar Murni seluruh data relasi sekaligus: **Material**, **Quiz**, **Assignment**, dan **PracticalLab (Lab)**.
4. **Fungsi `update` dan `remove`:** Ini sangat kritis. Sebelum memproses `update/delete`, ambil data `course` dan bandingkan `instructorId` dengan ID User dari token JWT. **ATURAN MUTLAK:** Jika berbeda, wajib tolak request dengan melemparkan error 403 `ForbiddenException`.

D. Uji Coba (Testing Postman)

1. Panggil `GET /courses`. Periksa JSON response; pastikan ada objek informasi instruktur dan total jumlah mahasiswa (dari fungsi `COUNT`).
2. Panggil `GET /courses/:id`. Pastikan 4 tab informasi sekaligus (Material, Quiz, Assignment, Lab) muncul dalam satu struktur JSON response.
3. Buat request `DELETE` ke course milik Anda yang sudah memiliki banyak data anak (materi, pendaftaran mahasiswa, dll).
4. Buka `npx prisma studio` dan buktikan bahwa seluruh data anak yang terkait dengan Course tersebut di semua tabel ikut musnah.
5. Coba hapus/edit course milik dosen lain untuk memastikan sistem menolak dengan status 403.

E. Selesai? Push ke GitHub

1. Masukkan semua perubahan Anda: `git add .`
2. Simpan sejarah pekerjaan Anda: `git commit -m "feat: comprehensive course relations and total cascade delete"`
3. Unggah ke GitHub server: `git push origin fitur/backend-sinta-course`
4. Buat Pull Request ke `develop` dan tugaskan Ariel sebagai reviewer.

3. 👤 TUGAS TIO — Quiz CRUD & Anti-Cheat System

Branch: `fitur/backend-tio-quiz`

Deskripsi

Anda memegang kendali atas modul evaluasi (Kuis). Tugas utama Anda adalah melengkapi alur manajemen kuis. Dua tantangan utama adalah menambahkan kemampuan penghitungan jumlah soal dinamis, dan merekayasa API agar kunci jawaban bersih (tidak bocor) sebelum JSON dikirimkan ke mahasiswa.

Langkah-Langkah

A. Persiapan Awal

1. Pindah ke branch utama (`git checkout develop`) lalu tarik update kode terbaru dari tim (`git pull origin develop`).
2. Buat branch terpisah khusus untuk tugas Anda (`git checkout -b fitur/backend-tio-quiz`).
3. Masuk ke folder backend (`cd apps/api`) lalu jalankan `npm install` untuk memastikan semua library/dependency terbaru terunduh dan sinkron.

B. Pembaruan Database (Prisma)

1. Buka file skema database di `apps/api/prisma/schema.prisma`.
2. Temukan model `QuizQuestion`. Pada baris relasi yang mengikat pertanyaan tersebut ke model `Quiz`, tambahkan properti hapus kaskade (`onDelete: Cascade`).
3. Jalankan `npx prisma generate` di terminal untuk memperbarui Prisma Client sehingga kode Anda bisa membaca aturan relasi Cascade yang baru.

C. Mulai Coding (Logic Lanjutan)

1. Buka file service kuis (`quiz.service.ts`).
2. Fungsi `findAll`:
 - Sediakan opsi filter berdasarkan parameter URL `courseId`.
 - Modifikasi struktur kembalian agar response array memuat `id`, judul, hadiah XP, standar lulus (`passingScore`), batas waktu, dan **jumlah total soal** (hitung menggunakan fungsi agregasi `_count` dari relasi `questions`).
3. Fungsi `findOne` (Proteksi Jawaban):
 - Lakukan query ke Prisma untuk menarik satu kuis beserta relasi soal-soalnya (`questions`).
 - **ATURAN KRUSIAL:** Anda dilarang keras mengembalikan field `correctAnswer` (kunci jawaban). Lakukan iterasi (perulangan) pada array pertanyaan, dan hapus/kecualikan properti kunci jawaban dari setiap objek sebelum mengembalikan data ke controller.
4. Fungsi `update` dan `remove`: Tambahkan pengecekan validasi kepemilikan. Pastikan hanya dosen pemilik kuis yang berwenang mengubah judul, batas waktu, `xpReward`, atau menghapus kuis.

D. Uji Coba (Testing Postman)

1. Panggil `GET /quizzes`. Pastikan hasil JSON mencantumkan angka jumlah total soal (hasil `COUNT`) untuk setiap kuis di dalam daftarnya.
2. Panggil `GET /quizzes/[id]`. Inspeksi response array soal dengan sangat teliti. Pastikan tidak ada satupun field kunci jawaban (`correctAnswer`) yang bocor ke publik.
3. Lakukan pengujian filter dengan `GET /quizzes?courseId=123` untuk memastikan sistem hanya memunculkan kuis dari kelas tersebut.
4. Coba skenario penghapusan kuis dan buktikan via Prisma Studio bahwa pertanyaan-pertanyaan di dalam tabel `QuizQuestion` ikut musnah secara otomatis.

E. Selesai? Push ke GitHub

1. Masukkan semua perubahan Anda: `git add .`
2. Simpan sejarah pekerjaan Anda: `git commit -m "feat: quiz counts integration and strict answer protection"`
3. Unggah ke GitHub server: `git push origin fitur/backend-tio-quiz`
4. Buat Pull Request ke `develop` dan tugaskan Ariel sebagai reviewer.

4. 👤 TUGAS DIMAS — Assignment CRUD & Deadline Logic

Branch: `fitur/backend-dimas-assignment`

Deskripsi

Tugas Anda adalah mematangkan modul penugasan (Assignment). Anda harus melengkapi fungsionalitas CRUD secara penuh. Tantangannya adalah mengintegrasikan presisi batas waktu (Deadline) yang ketat dan memastikan konfigurasi pembersihan otomatis (Cascade Delete) disiapkan untuk file jawaban mahasiswa (Submission) di masa mendatang.

Langkah-Langkah

A. Persiapan Awal

1. Pindah ke branch utama (`git checkout develop`) lalu tarik update kode terbaru dari tim (`git pull origin develop`).
2. Buat branch terpisah khusus untuk tugas Anda (`git checkout -b fitur/backend-dimas-assignment`).
3. Masuk ke folder backend (`cd apps/api`) lalu jalankan `npm install` untuk memastikan semua library/dependency terbaru terunduh dan sinkron.

B. Pembaruan Database (Prisma)

1. Buka file konfigurasi skema di `apps/api/prisma/schema.prisma`.
2. Cari definisi untuk model `Assignment`. Tambahkan field baru: `deadline DateTime` untuk menyimpan waktu pengumpulan akhir secara absolut.
3. Buat persiapan model baru `AssignmentSubmission` (jika belum ada) dan pastikan baris relasinya ke tabel `Assignment` diberikan instruksi khusus `onDelete: Cascade`. Ini krusial agar saat tugas dihapus oleh Dosen, seluruh file lampiran mahasiswa ikut musnah tak bersisa.
4. Jalankan `npx prisma migrate dev --name add_deadline_and_submission_cascade` di terminal untuk mengeksekusi perubahan ini ke database sungguhan dan menyimpan histori migrasinya.

C. Mulai Coding (Logic Lanjutan)

1. Buka `assignment.service.ts` beserta `assignment.controller.ts`.
2. **Fungsi `findAll`:**
 - Sediakan kapabilitas penyaringan data berdasarkan filter URL `courseId`.
 - Modifikasi agar sistem mengembalikan JSON yang memuat id tugas, judul, deskripsi, dan tanggal deadline secara akurat.
3. **Fungsi `findOne`:** Modifikasi endpoint pengambilan rincian tugas agar mengekspos instruksi detail dan field batas waktu (`deadline`) secara utuh. Keakuratan format tanggal ini krusial untuk rendering frontend.
4. **Fungsi `create / update`:** Saat menyimpan modifikasi instruksi atau perpanjangan waktu, pastikan string penanggalan dikonversi menjadi objek waktu yang sah (e.g., `new Date(dto.deadline)`).
5. **Fungsi `update dan remove`:** Terapkan validasi hak akses berjenjang. Anda harus memverifikasi bahwa ID dosen dari mata kuliah tersebut selaras dengan ID user dari token JWT. Tolak operasi dengan error jika bukan dosen pemilik.

D. Uji Coba (Testing Postman)

1. Buat request POST ke endpoint pembuatan penugasan baru. Di dalam tab "Body", sisipkan field `deadline` dengan format ISO yang akurat (contoh: `"2026-05-20T23:59:00Z"`).
2. Panggil rincian tugas (`GET /assignments/:id`) dan verifikasi bahwa tanggal `deadline` dapat dipanggil utuh tanpa korupsi tipe data.
3. Panggil `GET /assignments?courseId=xxx`. Pastikan sistem mampu mengelompokkan data dengan akurat.
4. Coba paksa menghapus tugas yang sudah memiliki relasi file jawaban mahasiswa (jika tabel `Submission` sudah diisi manual di Prisma Studio) dan buktikan mekanisme `Cascade Delete` bekerja membersihkan tabel `Submission` tersebut.

E. Selesai? Push ke GitHub

1. Masukkan semua perubahan Anda: `git add .`
2. Simpan sejarah pekerjaan Anda: `git commit -m "feat: complete assignment module with strict deadline tracking and cascade submission delete"`
3. Unggah ke GitHub server: `git push origin fitur/backend-dimas-assignment`
4. Buat Pull Request ke `develop` dan tugaskan Ariel sebagai reviewer.

Reminder Final

- Selalu gunakan pola eksekusi asinkron (`async/await`) saat membungkus panggilan operasi ke layanan database Prisma.
- Jika Anda mengalami Merge Conflict saat melakukan integrasi atau `git pull`, jangan mengambil risiko menyelesaikan konflik secara paksa jika Anda ragu. Segera komunikasikan di grup atau hubungi Leader.
- **Deadline Sprint 2:** Sabtu, 16 Mei 2026. Targetkan penyelesaian coding Anda sebelum tanggal tersebut untuk menyisakan waktu bagi proses peninjauan kode (code review).

Dokumen panduan ini disusun sedemikian rupa untuk memastikan sinkronisasi teknis mendalam dan pemahaman logika bisnis antar seluruh anggota tim backend Ruang Dosen.